



OPEN ACCESS

EDITED BY

Vasile Daniel Pavaloia,
Alexandru Ioan Cuza University, Romania

REVIEWED BY

Barkaoui Kamel,
Conservatoire National des Arts et
Métiers (CNAM), France
Rully Agus Hendrawan,
Sepuluh Nopember Institute of
Technology, Indonesia

*CORRESPONDENCE

Joseph Noel
✉ josephnoel.phdinds2024@aim.edu
Christopher Monterola
✉ cmonterola@aim.edu

RECEIVED 14 September 2025

REVISED 23 February 2026

ACCEPTED 27 February 2026

PUBLISHED 20 March 2026

CITATION

Noel J, Monterola C and Tan DS (2026)
Small LLMs can be good coldstart
recommenders.
Front. Artif. Intell. 9:1705245.
doi: 10.3389/frai.2026.1705245

COPYRIGHT

© 2026 Noel, Monterola and Tan. This is
an open-access article distributed under
the terms of the [Creative Commons
Attribution License \(CC BY\)](#). The use,
distribution or reproduction in other
forums is permitted, provided the
original author(s) and the copyright
owner(s) are credited and that the
original publication in this journal is
cited, in accordance with accepted
academic practice. No use, distribution
or reproduction is permitted which does
not comply with these terms.

Small LLMs can be good coldstart recommenders

Joseph Noel^{1*}, Christopher Monterola^{1*} and
Daniel Stanley Tan^{1,2}

¹Aboitiz School of Innovation, Technology and Entrepreneurship, Asian Institute of Management, Makati, Philippines, ²Faculty of Science, Open Universiteit Nederland, Heerlen, Limburg, Netherlands

Large Language Models (LLMs) have revolutionized the Artificial Intelligence (AI) field since the launch of ChatGPT in 2022. Since then, increasingly larger models have been released such as ChatGPT-4o having over 175 billion parameters, Llama 3.1 with 405 billion parameters, and PaLM with 560 billion parameters. However, LLMs of these sizes are no longer feasible to run easily outside of the largest research labs and organizations due to the extremely large amount of GPU compute required for both training and inference. More recently, research effort has been done to create smaller LLMs which can still perform relatively well compared to much larger models. Research has also been done to apply LLMs for domain-specific use cases such as recommendation systems via prompt engineering and fine-tuning. In this paper we combine the two research fields and fine-tune two small LLMs (2 billion parameters or less) for the sequential recommendation task. We find that fine-tuned small LLMs still perform as well and can even be better than standard sequential recommendation baseline models such as GRU4Rec and SASRec, especially in the coldstart setting.

KEYWORDS

coldstart recommendations, large language models, machine learning, PEFT, recommendation systems

1 Introduction

Pre-trained Large Language Models have been useful as foundation models which can then be customized for downstream tasks either via prompt engineering or via fine-tuning. They have been customized to domains such as music (Agostinelli et al., 2023), healthcare (Meng et al., 2024), education (Ma et al., 2023; Khalid et al., 2021; Haruna et al., 2017), and forecasting (Jin et al., 2024). LLMs have also been applied to the recommendation domain in numerous works (Sanner et al., 2023; Bao et al., 2023; Harte et al., 2023; Wei et al., 2024), taking advantage of their pre-trained learned representations and the expressivity of natural language. However most research of LLMs for recommendation systems make use of state-of-the-art LLMs with over 100 billion parameters. Training and running LLMs of these sizes are intractable for all but the largest and best funded organizations due to the extremely large amount of GPU compute required for both training and inference.

Small LLMs are becoming an active field of development (Wan et al., 2024) due to their cheaper compute cost and they have become more capable over time. These models are small enough that they can be fine-tuned and run using off-the-shelf GPUs which are more readily available to everyone.

In this paper we explore fine-tuning small LLMs with 2 billion parameters or less for the recommendation domain. We use Low-Rank Adaptation (LoRA) to fine-tune two small

LLMs, Danube-1.8B (Singer et al., 2024) and Gemma-2B (Team et al., 2024), and evaluate them on two standard recommendations datasets, MovieLens10M (Harper and Konstan, 2015) and Yoochoose-clicks (Ben-Shimon et al., 2015). We find that the fine-tuned LLMs are able to adequately learn to do sequential recommendation, and are able to beat the baseline recommendation models in the coldstart setting. To the best of our knowledge this is the first work on fine-tuning small LLMs for the sequential recommendation domain in a coldstart setting.

2 Background and related work

2.1 Recommendation systems

Let U be the set of users and X be the set of items. Recommendation systems use machine learning algorithms to predict a user-item rating $R(u, x)$ for all users $u \in U$ and all items $x \in X$.

Different classes of recommendation system models have been developed over the years, such as Content-based Filtering (Lops et al., 2019), Collaborative Filtering (Zhang et al., 2014; Salakhutdinov and Mnih, 2007; Linden et al., 2003), and Sequential Recommendation (Wu et al., 2019; Li et al., 2017; Liu et al., 2018; Yu et al., 2020; Hidasi and Karatzoglou, 2018; Wang-Cheng Kang, 2018). Availability of past user preference information is a concern as model training is mainly done on previous user-item interactions such as explicit rating scores and implicit activities such as item clicks or views.

2.1.1 Sequential recommendation

A common formulation of the recommendation problem models the data as a sequence of user-item interactions. In sequential recommendation, let X be the set of items to be recommended, and $x_{1:t} = x_1, x_2, \dots, x_t$ be the sequence of past user-item interactions where $x_i \in X$ is the user-item interaction at timestamp t . A sequential recommendation model can be a multi-class classifier where, given the interaction sequence $x_{1:t}$, the model tries to predict the next item in the sequence x_{t+1} . The model output can be a ranked list of items with classification logits $y_{t+1} = [y_1, y_2, \dots, y_n] \in \mathbb{R}^n$ where $n = |I|$ is the number of possible items. The final recommendation list at timestamp $t+1$ are the top- k items from y_{t+1} .

2.1.2 Coldstart recommendation

The coldstart problem is one of the fundamental research problem in the field of recommendation systems and numerous research efforts have been dedicated to solving it (Gope and Jain, 2017). Because traditional recommendation models rely on past user-item interactions, new users and items won't have had enough historical information to make accurate recommendation predictions on. New items in particular may be disadvantaged and won't be recommended at all due to popularity bias which can favor older items (Noel et al., 2024; Abdollahpouri et al., 2017).

Popular methods devised for handling the coldstart problem include incorporating user and item attributes in the model training (Gantner et al., 2010; Burke, 2007), hybrid content-based and collaborative filtering algorithms (Schein et al., 2002; Stern et al., 2009), user classification (Lika et al., 2014), cross-domain recommendation (Kang et al., 2019; Man et al., 2017; Omidvar and Tran, 2023), and novel objective functions and regularization (Wei et al., 2021; Abdollahpouri et al., 2017; Kuznetsov and Kordík, 2023).

With the advent of LLMs, new methods for data augmentation (Wei et al., 2024) and initial preference elicitations (Sanner et al., 2023) have also been explored for coldstart recommendations.

2.2 Large language model recommendation systems

Numerous works have previously explored the use of language models for recommendation. One such use is as a single unifying model architecture that can handle different recommendation problems such as sequential recommendation, ratings prediction, and review summarization (Harte et al., 2022). Others have used bi-directional encoders pioneered in BERT (Devlin et al., 2019) for recommendation (Zhang et al., 2019; Chen et al., 2019).

The advent of pre-trained Large Language Models has brought about numerous research investigating their applicability for recommendation. Most applications have focused on their usefulness on the coldstart problem. LLMs from OpenAI have been used for data augmentation to handle the common coldstart and data sparsity challenge in recommendation systems (Wei et al., 2024). OpenAI's LLM text embeddings have also been used to initialize BERT4Rec (Zhang et al., 2019) item embeddings and have been found to improve its performance (Harte et al., 2023). Google's PaLM (Chowdhery et al., 2023) was used to investigate solely using prompt-engineering for recommendation and found it was competitive in near cold-start settings (Sanner et al., 2023). Facebook's Llama (Touvron et al., 2023a) was fine-tuned for a binary recommendation problem and performed well in a few-shot setting which is similar to coldstart (Bao et al., 2023).

The above examples all use state-of-the-art and extremely large language models with the order of hundreds of billions of parameters. Running these LLMs locally will require extremely large amounts of compute which may not be feasible for most organizations. We explore two options for scaling down the requirements of using LLMs for recommendation: Smaller large language models and parameter-efficient fine-tuning.

2.2.1 Small large language models

Recently, research has been done on the creation of smaller large language models which can still perform competitively with the much larger state-of-the-art LLMs (Wan et al., 2024; Sheik et al., 2024). Smaller models are easier to fine-tune and will have cheaper inference costs when deployed in real-world production use-cases (Singer et al., 2024; Team et al., 2024; Zhang et al., 2024). We note that the term "small" here is relative and applied specifically to

LLMs which have less than 2 billion parameters, which can still be quite large when compared to more traditional machine learning models.

2.2.2 LLM fine-tuning

Fine-tuning is a commonly used technique in deep learning to take advantage of pre-trained models and adapt them for other domains (Jung et al., 2015; Yin et al., 2017; Jaques et al., 2016, 2017; Howard and Ruder, 2018). Due to the extremely large model sizes of LLMs, updating the weights of the entire model is infeasible without having access to a large amount of GPU compute which may be unrealistic for most organizations and researchers. Techniques for parameter-efficient fine-tuning (PEFT) (Ding et al., 2023) needed to be developed.

One of the more popular PEFT methods is Low-Rank Adaptation (LoRA) (Hu et al., 2022). LoRA works by adding a small number of new weight matrices into the model and only these new additions are updated during fine-tuning training. The much smaller number of trainable parameters makes the fine-tuning process faster and more efficient compared to updating all the weights of the entire model. Formally, the learning objective for an LLM can be defined as

$$\max_{\Phi} \sum_{(x,y) \in Z} \sum_{t=1}^{|y|} \log(P_{\Phi}(y_t|x, y_{<t})),$$

where x and y are in the input and output tokens of training set Z , y_t is the t -th token of y , $y_{<t}$ are the tokens before y_t , and Φ is the original parameter weights. The number of parameters in Φ will be very large for large language models, therefore LoRA introduces a new set of parameters Θ such that

$$\max_{\Theta} \sum_{(x,y) \in Z} \sum_{t=1}^{|y|} \log(P_{\Phi+\Theta}(y_t|x, y_{<t})),$$

and where the number of new parameters in Θ will be much smaller than the number of parameters in Φ . Only the parameters of Θ are updated during the LoRA fine-tuning, making this more efficient and less time-consuming than regular fine-tuning.

In the next section we fine-tune small LLMs for the sequential recommendation domain in the coldstart setting using LoRA and show our results.

3 Experiments

3.1 Dataset

We evaluate our method using two datasets: Yoochoose-clicks (Ben-Shimon et al., 2015), and MovieLens10M (Harper and Konstan, 2015). In MovieLens10M each user-movie rating is considered an interaction, with the actual value of the rating being disregarded. In Yoochoose-clicks we remove duplicate item IDs from the sequences. We also follow a common practice of sampling

TABLE 1 Number of sequences used for training and testing.

Dataset	Training sequences	Test Sequences
Yoochoose	53,840	13,560
MovieLens	55,902	13,976

only a fraction of the data due to its large size (Wu et al., 2019; Li et al., 2017; Liu et al., 2018; Yu et al., 2020).

For both datasets we use the last interaction in the sequence as the prediction target x_{t+1} , and only get the previous 5 interactions as the input sequence. This leaves a limited set of interactions which simulates the coldstart setting, where only a small number of users will have a limited number of interactions with the items. Table 1 shows the number of sequences used for training and testing in our experiments. Only the item IDs are used from the dataset and no other additional user or item features have been utilized.

3.1.1 LLM fine-tuning dataset

To fine-tune the LLMs we need to convert the datasets into prompts suitable for causal language modeling. Causal language modeling is a text generation workflow which predicts the next token in a sequence of tokens, using only the previous tokens as information. We convert the sequence input and target output into prompts in the following manner shown in Table 2. The final word target output is left out of the test input data and the LLM is prompted for the predicted output.

3.2 Small LLMs

We use two open-source LLM models in our experiments, Danube-1.8B (Singer et al., 2024) and Gemma-2B (Team et al., 2024). Danube-1.8B is a decoder model based on the Llama 2 architecture (Touvron et al., 2023b) with 1.8 billion parameters and trained on 1 trillion tokens. Gemma-2B is decoder model with 2 billion parameters and trained on 3 trillion tokens. A key element in selecting these models is their open-source nature so that their trained model weights are available online^{1,2} and can be used for further fine-tuning.

We use LoRA (Hu et al., 2022) for efficient fine-tuning of both models for recommendation, using the converted prompts dataset detailed in the previous section. Table 3 shows the number of trainable parameters that were used for fine-tuning and what percentage they were of the original model size. It can be seen that we are updating only a very small fraction (less than 1%) of the number of weights of the original models, which allows us to fine-tune them on relatively dated NVIDIA GeForce GTX 1080 Ti GPUs.

1 <https://huggingface.co/h2oai/h2o-danube-1.8b-base>

2 <https://huggingface.co/google/gemma-2b>

TABLE 2 LLM fine-tuning dataset converted to prompts.

Dataset	Example
Training data	"This user interacted with the following items in the given order: 466, 520, 151, 1408, 1912. The next movie the user will click is 8784"
Test data	"This user interacted with the following items in the given order: 832, 1271, 3108, 3252, 224. The next item the user will click is"
Target output	"5629"

TABLE 3 Number of trainable LoRA parameters and their percentage of the original model size.

LLM	Trainable parameters
Danube	8,650,752 (0.47%)
Gemma	9,805,824 (0.39%)

3.2.1 Baseline models

We compare against three deep learning sequential recommendation models as baselines, GRU4Rec (Hidasi et al., 2016), SASRec (Wang-Cheng Kang, 2018) and BERT4Rec (Zhang et al., 2019). All models are trained with the Adam (Kingma and Ba, 2015) optimizer using cross-entropy loss. Model hyperparameters were tuned with grid-search and the best results are reported.

The details of their final model architectures are below:

- **GRU4Rec:** 1 embedding layer and 1 GRU layer, 20% dropout and fully-connected dense output layer with softmax output. The embedding and GRU layers have sizes of both 100 for MovieLens and sizes of 400 and 100 for Yoochoose.
- **SASRec:** 1 embedding layer and 1 self-attention layer, 20% dropout and fully-connected dense output layer with softmax output. The embedding sizes and feedforward layer sizes are 64 and 256 for MovieLens and 32 and 512 for Yoochoose.
- **BERT4Rec:** 1 embedding layer and 2 self-attention layers, 20% dropout and fully-connected dense output layer with softmax output. The embedding sizes and feedforward layer sizes are 64 and 128 for MovieLens and 64 and 64 for Yoochoose.

3.2.2 Metrics

To measure the performance of our model we use HitRate@1. HitRate@ k measures whether the correct item is in the top- k position in the recommendation list. HitRate@1 is equivalent to multi-class classification accuracy.

We also make use of two additional metrics to help us analyze the predicted outputs of the models. Given that the LLMs are predicting the item IDs by each single-digit token one by one, we measure the *average deviation* and the *average distance* of the predicted item IDs to the IDs in the input sequence. We measure the average deviation by getting the absolute difference between the predicted ID with each input ID and averaging them. In our experiments where $N = 5$, y is the prediction output and x_i is the i -th item ID in the input sequence, the average deviation is calculated as:

$$\sum_i^N \frac{|y - x_i|}{N}$$

The average distance is calculated using the hamming distance between two strings which is the number of positions in the strings in which they differ. For example the strings "11059" and "15069" have a hamming distance of 2, pertaining to the two positions that are bolded. The average distance is calculated as:

$$\sum_i^N \frac{\text{Hamming}(y, x_i)}{N}$$

4 Results

We run our experiments five times for each model and dataset combination and the average of the results are presented. We also do one-way ANOVA (Fisher, 1992) followed by Tukey's HSD test (Tukey, 1949) to confirm that the difference in results of the LLMs compared to the baseline methods are statistically significant.

Table 4 shows the HitRate@1 of the LLMs together with the baseline models, along with the average distance and average deviation of their predicted outputs. We see that the LLMs have superior sequential recommendation performance in the MovieLens and Yoochoose datasets under the coldstart setting. With the limited training data, the LLMs were able to take advantage of the closer similarity between IDs in the sequences for its predictions. The baseline algorithms are trying to learn embeddings for each item ID which necessitates more training data to model the user preferences accurately.

Finally, in both datasets the average distance and deviation of the LLMs are smaller than of the baseline models. This suggests that the LLMs are more likely to predict item ID values that are closer in value to the input sequence both textually and numerically. Sequences of IDs that are relatively close to each other are easier for the LLM to learn efficiently.

5 Analysis

5.1 Tokenization behavior

We analyzed how both Gemma-2B and Danube-1.8B tokenize item IDs. As shown in Table 5 neither model treats numeric item IDs as atomic symbols. Instead, each ID is decomposed into a sequence of digit-level tokens. Because the number of tokens corresponds directly to the number of digits, the models implicitly learn the morphological structure of item IDs rather than their symbolic identity. This explains why the LLMs sometimes predict outputs with similar digit patterns or similar number of digits.

However this tokenizer-induced numeric bias cannot fully account for model behavior. Many correct predictions correspond to items that are not numerically close to any input IDs. These predictions cannot be explained by digit continuity or numeric similarity, indicating that the models are also learning co-occurrence structure in the data rather than performing trivial

TABLE 4 Average distance and average deviation of predictions.

Dataset	Model	HitRate@1	Average distance	Average deviation
Yoochoose	Danube	0.0555	3.50	11,610.96
	Gemma	0.0540	3.48	11,326.80
	GRU4Rec	0.0440	3.75	13,336.70
	SASRec	0.0354	3.82	13,906.85
	BERT4Rec	0.0499	3.66	12,419.45
MovieLens	Danube	0.0995	3.20	4163.35
	Gemma	0.1019	3.19	3,999.54
	GRU4Rec	0.0934	3.35	4,773.73
	SASRec	0.0898	3.37	5,319.92
	BERT4Rec	0.0854	3.34	4,429.16

Highest HR@1 and the lowest Average Distance and Average Deviation are in bold. The LLMs perform better than the baseline algorithms in the coldstart setting and their predictions are closer to the input sequences both textually and numerically.

TABLE 5 Example of tokenization output for both Danube and Gemma.

Model	ID	Tokenization
Danube	4,568	[4,5,6,8]
	376	[3,7,7]
Gemma	4,568	[4,5,6,8]
	376	[3,7,7]

TABLE 6 Sample of correct predictions which are not numerically close to the input IDs.

Input	Prediction
36,431, 40,655, 40,661, 249, 28,858	30,761
421, 1,183, 205, 143, 117	722
50, 47, 51, 28,413, 9	23,164
2,614, 41,456, 3,735, 6,775, 41,446	2,210

numeric continuations. These observations suggest that small LLMs do exploit digit-level regularities imposed by tokenization but also learn meaningful sequential patterns beyond simple numeric morphology. We show a sample of these predictions from the Danube model in Table 6.

5.2 Input Sequence Length

We also ran additional experiments on the Movielens dataset using the Danube-1.8B LLM model and the GRU4Rec baseline model to test the effects of increasing the length of input sequences and show these results in Table 7. Unlike conventional sequential recommenders which typically benefit from longer interaction histories, we find that the small LLM model was not able to take advantage of the longer input sequences to improve its performance significantly, whereas the GRU4Rec model was able to better improve its performance as the input sequence length increased.

Our results show that while the baseline GRU4Rec model can better leverage longer histories through learned item

TABLE 7 Results of different input sequence lengths on the Movielens dataset.

Model	Input sequence length	Tokenization
Danube	5	0.0995
	10	0.1044
	20	0.0973
	50	0.0917
GRU4Rec	5	0.0934
	10	0.0961
	20	0.1037
	50	0.1080

Best result in bold.

representations, the small LLMs shine at shorter item interaction histories typical of the coldstart scenario in our study.

5.3 Inference efficiency

We also measured inference performance for the small LLMs used in our experiments. In our machines,³ Gemma-2B achieved a latency of 59.5ms per generated token, while Danube-1.8B achieved 34.3ms, with both models using approximately 5.2GB of GPU memory during inference. Larger LLMs will require much larger computing requirements and also higher latency in the same machines (Chitty-Venkata et al., 2025). Additionally, while these latencies are higher than those of the baseline recommendation models, the LLMs do not require a separate item-embedding matrix whose size grows linearly with the number of items in the catalog. LLMs operate directly on the tokenized string representation of item IDs and therefore rely on a fixed-size tokenizer vocabulary and embedding table. As a result, the memory footprint of the LLMs remains constant regardless of catalog size which can be advantageous in domains where item vocabularies are large or dynamic.

6 Conclusion

We have shown that small LLMs can punch above their weight and be viable sequential recommendation models after fine-tuning with LoRA. We also found that the LLMs were more likely to make predictions that are closer to the input sequences both textually and numerically. In our experiments the small LLMs were even able to beat standard sequential recommendation models in the coldstart setting. We also performed token-level analysis, which showed that although the models exploit digit-level tokenization patterns which biases numeric proximity, they also learn meaningful sequential patterns. The LLMs also avoid item-embedding tables and therefore scale independently of catalog size which can be advantageous in domains with extremely large vocabulary sizes. Finally, we also looked at the effects of longer input sequence histories and found that traditional sequential recommenders like GRU4Rec are able

³ Intel Xeon CPU with 4 NVIDIA GeForce RTX 2080 Ti GPUs.

to leverage longer item histories better and will eventually beat the small LLMs outside of the coldstart scenario. A future study beyond the scope of this paper could be on how to improve the small LLMs to better leverage longer item histories.

Our results holds great promise in future democratization of the use of small LLMs, which can be run by more organizations with relatively lesser compute capacities. Future work in this field can explore even smaller LLMs, as well as their applicability for other recommendation domains such as ratings predictions. A more systematic study of tokenization strategies such as learned ID embeddings or alternative numeric decomposition methods can be also done for comparison. Using small LLMs for feature augmentation or dataset augmentation can also be additional use cases for exploration. These directions can deepen our understanding of when small LLMs represent a practical alternative to embedding-based recommenders and how they can be used most effectively within real-world systems. We hope that our work also encourages the use of open-source and smaller LLMs in other fields outside the recommendation domain, as they may provide a more realistic alternative than always going for the biggest state-of-the-art models.

Data availability statement

The original contributions presented in the study are included in the article/supplementary material, further inquiries can be directed to the corresponding author.

Author contributions

JN: Writing – original draft, Writing – review & editing. CM: Supervision, Writing – review & editing. DT: Supervision, Writing – review & editing.

References

- Abdollahpouri, H., Burke, R., and Mobasher, B. (2017). “Controlling popularity bias in learning-to-rank recommendation,” in *Proceedings of the 11th ACM Conference on Recommender Systems*.
- Agostinelli, A., Denk, T. I., Borsos, Z., Engel, J., Verzetti, M., Caillon, A., et al. (2023). *Musichm: Generating Music From Text*. arXiv [Preprint]. arXiv:2301.11325.
- Bao, K., Zhang, J., Zhang, Y., Wang, W., Feng, F., and He, X. (2023). “TALLRec: an effective and efficient tuning framework to align large language model with recommendation,” in *Proceedings of the 17th ACM Conference on Recommender Systems* (New York, NY: Association for Computing Machinery).
- Ben-Shimon, D., Tsikinovsky, A., Friedmann, M., Shapira, B., Rokach, L., and Hoerle, J. (2015). “RecSys challenge 2015 and the yoochoose dataset,” in *Proceedings of the 9th ACM Conference on Recommender Systems*.
- Burke, R. (2007). *Hybrid Web Recommender Systems*. Cham: Springer-Verlag, p. 377–408.
- Chen, X., Liu, D., Lei, C., Li, R., Zha, Z.-J., and Xiong, Z. (2019). “BERT4SessRec: Content-based video relevance prediction with bidirectional encoder representations from transformer,” in *Proceedings of the 27th ACM International Conference on Multimedia* (New York, NY: Association for Computing Machinery).
- Chitty-Venkata, K. T., Raskar, S., Kale, B., Ferdaus, F., Tanikanti, A., Raffanetti, K., et al. (2025). “LLM-inference-bench: inference benchmarking of large language models on ai accelerators,” in *Proceedings of the 2024 Workshops of the International Conference*

Funding

The author(s) declared that financial support was not received for this work and/or its publication.

Conflict of interest

The author(s) declared that this work was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

Generative AI statement

The author(s) declared that generative AI was not used in the creation of this manuscript.

Any alternative text (alt text) provided alongside figures in this article has been generated by Frontiers with the support of artificial intelligence and reasonable efforts have been made to ensure accuracy, including review by the authors wherever possible. If you identify any issues, please contact us.

Publisher's note

All claims expressed in this article are solely those of the authors and do not necessarily represent those of their affiliated organizations, or those of the publisher, the editors and the reviewers. Any product that may be evaluated in this article, or claim that may be made by its manufacturer, is not guaranteed or endorsed by the publisher.

on *High Performance Computing, Network, Storage, and Analysis* (Atlanta, GA: IEEE Press).

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., et al. (2023). PaLM: Scaling language modeling with pathways. *J. Mach. Learn. Res.* 24, 1–113. doi: 10.5555/3648699.3648939

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics* (Minneapolis, MN: Association for Computational Linguistics).

Ding, N., Qin, Y., Yang, G., Wei, F., Zonghan, Y., Su, Y., et al. (2023). Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nat. Mach. Intellig.* 5, 1–16. doi: 10.1038/s42256-023-00626-4

Fisher, R. (1992). “Statistical methods for research workers,” in *Breakthroughs in Statistics: Methodology and Distribution* (New York, NY: Springer).

Gantner, Z., Drumond, L., Freudenthaler, C., Rendle, S., and Schmidt-Thieme, L. (2010). “Learning attribute-to-feature mappings for cold-start recommendations,” in *2010 IEEE International Conference on Data Mining* (Sydney: IEEE).

Gope, J., and Jain, S. K. (2017). “A survey on solving cold start problem in recommender systems,” in *2017 International Conference on Computing, Communication and Automation (ICCCA)* (Greater Noida: IEEE), 133–138.

- Harper, F. M., and Konstan, J. A. (2015). The movielens datasets: history and context. *ACM Trans. Interact. Intellig. Syst.* 5, 1–19. doi: 10.1145/2827872
- Harte, J., Zörgdrager, W., Panos Louridas, A. K., Jannach, D., and Fragkoulis, M. (2022). “Shijie Geng and Shuchang Liu and Zuohui Fu and Yingqiang Ge Andyongfeng Zhang,” in *Proceedings of the 16th ACM Conference on Recommender Systems* (New York, NY: Association for Computing Machinery).
- Harte, J., Zörgdrager, W., Panos Louridas, A. K., Jannach, D., and Fragkoulis, M. (2023). “Leveraging large language models for sequential recommendation,” in *Proceedings of the 17th ACM Conference on Recommender Systems*.
- Haruna, K., Ismail, M. A., Damiasih, D., Sutopo, J., and Herawan, T. (2017). A collaborative approach for research paper recommender system. *PLoS ONE*. 12:e0184516. doi: 10.1371/journal.pone.0184516
- Hidasi, B., and Karatzoglou, A. (2018). “Recurrent neural networks with top-k gains for session-based recommendations,” in *Proceedings of the 27th ACM International Conference on Information and Knowledge Management* (New York, NY: Association for Computing Machinery).
- Hidasi, B., Karatzoglou, A., Baltrunas, L., and Tikk, D. (2016). “Session-based recommendations with recurrent neural networks,” in *Proceedings of the 4th International Conference on Learning Representations*.
- Howard, J., and Ruder, S. (2018). “Universal language model fine-tuning for text classification,” in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics* (Melbourne, VIC: Association for Computational Linguistics).
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., et al. (2022). “LoRA: Low-rank adaptation of large language models,” in *Proceedings of the 10th International Conference on Learning Representations*.
- Jaques, N., Gu, S., Bahdanau, D., Hernandez, J. M., Turner, L. R. E., and Eck, D. (2017). “Tuning recurrent neural networks with reinforcement learning,” in *Proceedings of the 5th International Conference on Learning Representations* (Toulon).
- Jaques, N., Gu, S., Turner, R. E., and Eck, D. (2016). “Generating music by fine-tuning recurrent neural networks with reinforcement learning,” in *Proceedings of the 30th Conference on Neural Information Processing Systems*.
- Jin, M., Wang, S., Ma, L., Chu, Z., Zhang, J. Y., Shi, X., et al. (2024). “Time-LLM: Time series forecasting by reprogramming large language models,” in *Proceedings of the 12th International Conference on Learning Representations* (Vienna).
- Jung, H., Lee, S., Yim, J., Park, S., and Kim, J. (2015). “Joint fine-tuning in deep neural networks for facial expression recognition,” in *Proceedings of the 2015 IEEE International Conference on Computer Vision* (Santiago: IEEE).
- Kang, S., Hwang, J., Lee, D., and Yu, H. (2019). “Semi-supervised learning for cross-domain recommendation to cold-start users,” in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. (New York, NY: Association for Computing Machinery).
- Khalid, A., Lundqvist, K., Yates, A., and Ghzanfar, M. A. (2021). Novel online recommendation algorithm for massive open online courses (nor-moocs). *PLoS ONE*. 16:e0245485. doi: 10.1371/journal.pone.0245485
- Kingma, D. P., and Ba, J. (2015). “Adam: a method for stochastic optimization,” in *Proceedings of the 3rd International Conference on Learning Representations* (San Diego, CA).
- Kuznetsov, S., and Kordik, P. (2023). Improving recommendation diversity and serendipity with an ontology-based algorithm for cold start environments. *Int. J. Data Sci. Analytics*. 20, 431–443.
- Li, J., Ren, P., Chen, Z., Ren, Z., Lian, T., and Ma, J. (2017). “Neural attentive session-based recommendation,” in *Proceedings of the 26th ACM Conference on Information and Knowledge Management* (New York, NY: Association for Computing Machinery).
- Lika, B., Kolomvatsos, K., and Hadjiefthymiades, S. (2014). Facing the cold start problem in recommender systems. *Expert Syst. Appl.* 41, 2065–2073. doi: 10.1016/j.eswa.2013.09.005
- Linden, G., Smith, B., and York, J. (2003). Amazon.com recommendations: item-to-item collaborative filtering. *IEEE Intern. Comp.* 7, 76–80. doi: 10.1109/MIC.2003.1167344
- Liu, Q., Zeng, Y., Mokhosi, R., and Zhang, H. (2018). “Stamp: Short-term attention/memory priority model for session-based recommendation,” in *Proceedings of the 24th ACM Conference on Knowledge Discovery and Data Mining* (New York, NY: Association for Computing Machinery).
- Lops, P., Jannach, D., Musto, C., Bogers, T., and Koolen, M. (2019). Trends in content-based recommendation. *User Model. User-Adapt. Interact.* 29, 239–249. doi: 10.1007/s11257-019-09231-w
- Ma, Y., Ouyang, R., Long, X., Gao, Z., Lai, T., and Fan, C. (2023). Doris: Personalized course recommendation system based on deep learning. *PLoS ONE*. 18:e0284687. doi: 10.1371/journal.pone.0284687
- Man, T., Shen, H., Jin, X., and Cheng, X. (2017). “Cross-domain recommendation: an embedding and mapping approach,” in *Proceedings of the 26th International Joint Conference on Artificial Intelligence* (AAAI Press).
- Meng, X., Yan, X., Zhang, K., Liu, D., Cui, X., Yang, Y., et al. (2024). The application of large language models in medicine: a scoping review. *iScience* 27:109713. doi: 10.1016/j.isci.2024.109713
- Noel, J., Monterola, C., and Tan, D. S. (2024). Improving recommendation diversity without retraining from scratch. *Int. J. Data Sci. Analytics*. 20, 1151–1160. doi: 10.1007/s41060-024-00518-9
- Omidvar, S., and Tran, T. (2023). Tackling cold-start with deep personalized transfer of user preferences for cross-domain recommendation. *Int. J. Data Sci. Analytics*. 20, 121–130. doi: 10.1007/s41060-023-00467-9
- Salakhutdinov, R., and Mnih, A. (2007). “Probabilistic matrix factorization,” in *Proceedings of the 20th International Conference on Neural Information Processing Systems* (Red Hook, NY: Curran Associates Inc.).
- Sanner, S., Balog, K., Radlinski, F., Wedin, B., and Dixon, L. (2023). “Large language models are competitive near cold-start recommenders for language and item-based preferences,” in *Proceedings of the 17th ACM Conference on Recommender Systems* (New York, NY: Association for Computing Machinery).
- Schein, A. I., Popescul, A., Ungar, L. H., and Pennock, D. M. (2002). “Methods and metrics for cold-start recommendations,” in *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development In Information Retrieval* (New York, NY: Association for Computing Machinery).
- Sheik, R., Sundara, K. P. S., and Nirmala, S. J. (2024). Neural data augmentation for legal overruling task: Small deep learning models vs. large language models. *Neural Proc. Letters* 56:4. doi: 10.1007/s11063-024-11574-4
- Singer, P., Pfeiffer, P., Babakhin, Y., Jeblick, M., Dhankhar, N., Fodor, G., et al. (2024). *H2o-danube-1.8b Technical Report*. arXiv [Preprint]. arXiv:2401.16818. doi: 10.48550/arXiv.2401.16818
- Stern, D., Herbrich, R., and Graepel, T. (2009). “Matchbox: large scale bayesian recommendations,” in *Proceedings of the 18th International World Wide Web Conference* (New York, NY: Association for Computing Machinery).
- Team, G., Mesnard, T., Hardin, C., Dadashi, R., Bhupatiraju, S., Pathak, S., et al. (2024). *Gemma: Open Models Based on Gemini Research and Technology*. arXiv [Preprint]. arXiv:2403.08295.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., et al. (2023a). *Llama: Open and Efficient Foundation Language Models*. arXiv [Preprint]. arXiv:2302.13971.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., et al. (2023b). *Llama 2: Open Foundation and Fine-Tuned Chat Models*. arXiv [Preprint]. arXiv:2307.09288.
- Tukey, J. (1949). Comparing individual means in the analysis of variance. *Biometrics* 5, 99–114. doi: 10.2307/3001913
- Wan, Z., Wang, X., Liu, C., Alam, S., Zheng, Y., Liu, J., et al. (2024). “Efficient large language models: A survey,” in *ACM Transactions on Interactive Intelligent Systems* (New York, NY: Association for Computing Machinery).
- Wang-Cheng Kang, J. M. (2018). “Self-attentive sequential recommendation,” in *Proceedings of the 18th IEEE International Conference on Data Mining* (Singapore: IEEE).
- Wei, W., Ren, X., Tang, J., Wang, Q., Su, L., Cheng, S., et al. (2024). “LLMRec: Large language models with graph augmentation for recommendation,” in *Proceedings of the 17th ACM International Conference on Web Search and Data Mining* (New York, NY: Association for Computing Machinery).
- Wei, Y., Wang, X., Li, Q., Nie, L., Li, Y., Li, X., et al. (2021). “Contrastive learning for cold-start recommendation,” in *Proceedings of the 29th ACM International Conference on Multimedia*. doi: 10.1145/3474085.3475665
- Wu, S., Tang, Y., Zhu, Y., Wang, L., Xie, X., and Tan, T. (2019). “Session-based recommendation with graph neural networks,” in *Proceedings of the 33rd AAAI Conference on Artificial Intelligence* (Washington, DC: AAAI Press).
- Yin, X., Chen, W., Wu, X., and Yue, H. (2017). “Fine-tuning and visualization of convolutional neural networks,” in *Proceedings of the 12th IEEE Conference on Industrial Electronics and Applications* (Siem Reap: IEEE).
- Yu, F., Zhu, Y., Liu, Q., Wu, S., Wang, L., and Tan, T. (2020). “Tagnn: Target attentive graph neural networks for session-based recommendation,” in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. (New York, NY: Transactions on Machine Learning Research).
- Zhang, P., Zeng, G., Wang, T., and Lu, W. (2024). *TinyLlama: An Open-Source Small Language Model*. arXiv [Preprint]. arXiv:2401.02385.
- Zhang, R., dong Liu, Q., Chun-Gui, W., Jia-Xuan, J.-X., and Huiyi-Ma (2014). “Collaborative filtering for recommender systems,” in *Proceedings of the 2nd International Conference on Advanced Cloud and Big Data* (Washington, DC: IEEE Computer Society).
- Zhang, R., dong Liu, Q., Chun-Gui, W., Jia-Xuan, J. X., and Huiyi-Ma (2019). “BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer,” in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management* (New York, NY: Association for Computing Machinery).